

Recommender Systems

COMPSCI 753



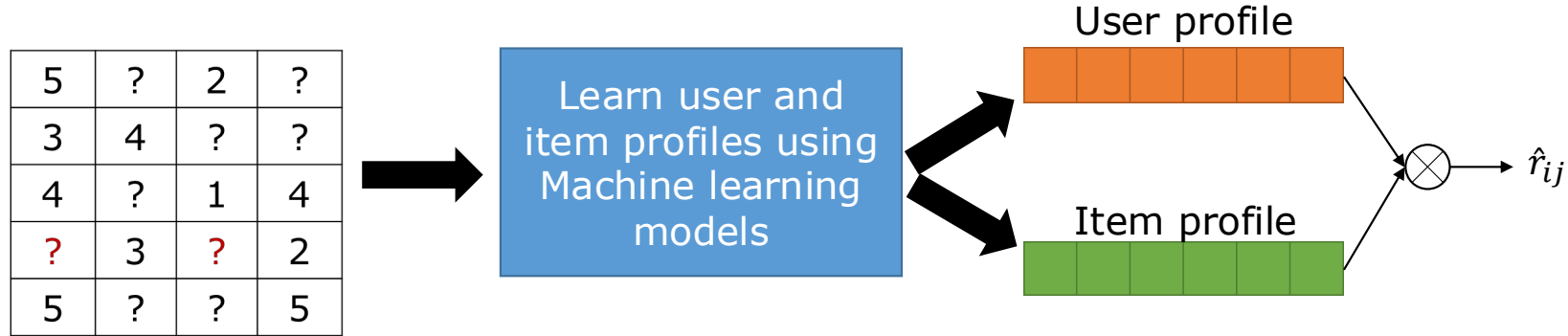
THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

Memory-based collaborative filtering

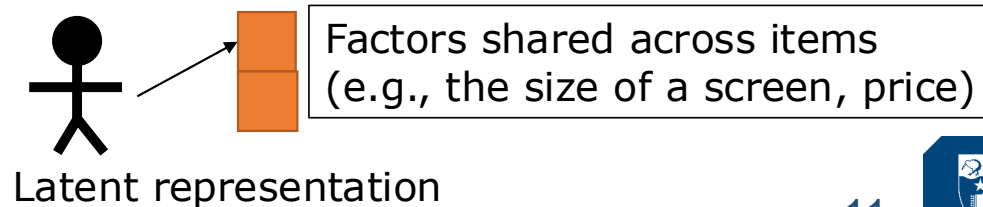
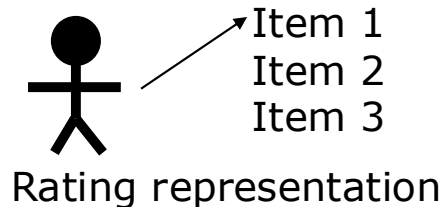
- So far, we have learned memory-based CF
 - Use the entire user-item rating matrix
 - Find nearest neighbors based on the ratings / interpolation weights
- **Challenges:**
 - Sparsity: <1% on real datasets
 - Each user often purchases hundreds from millions of items.
 - The similarity computed from the sparse user rating vector r_i results in poor similarity measure
 - Need dimension reduction
 - Nearest neighbor is not scalable
 - Computation cost grow with number of users and items, i.e., $O(nm)$
 - Need to store all the training transactions in memory

Model-based collaborative filtering

- Learn a model to uncover user preference to avoid searching nearest neighbors

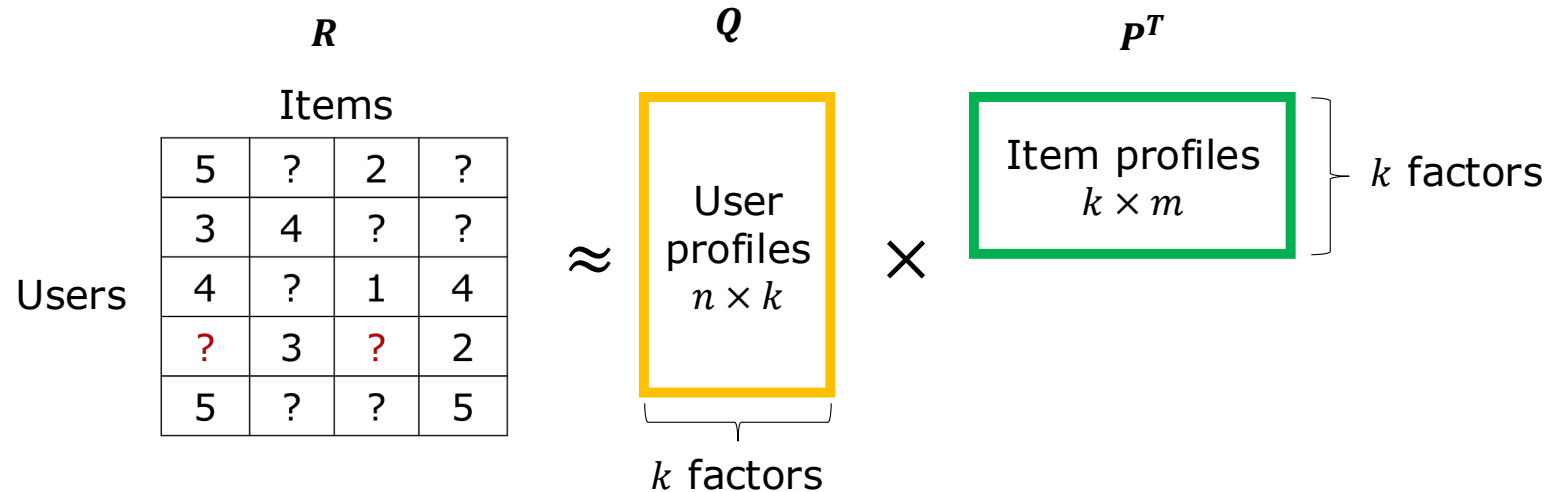


- Dimension reduction → handle sparse data



Matrix Factorization

- MF assumes there are latent vector representation of users and items.
- The latent vector of users and items can reconstruct the rating matrix



Example

- Netflix – (# factors = 3)

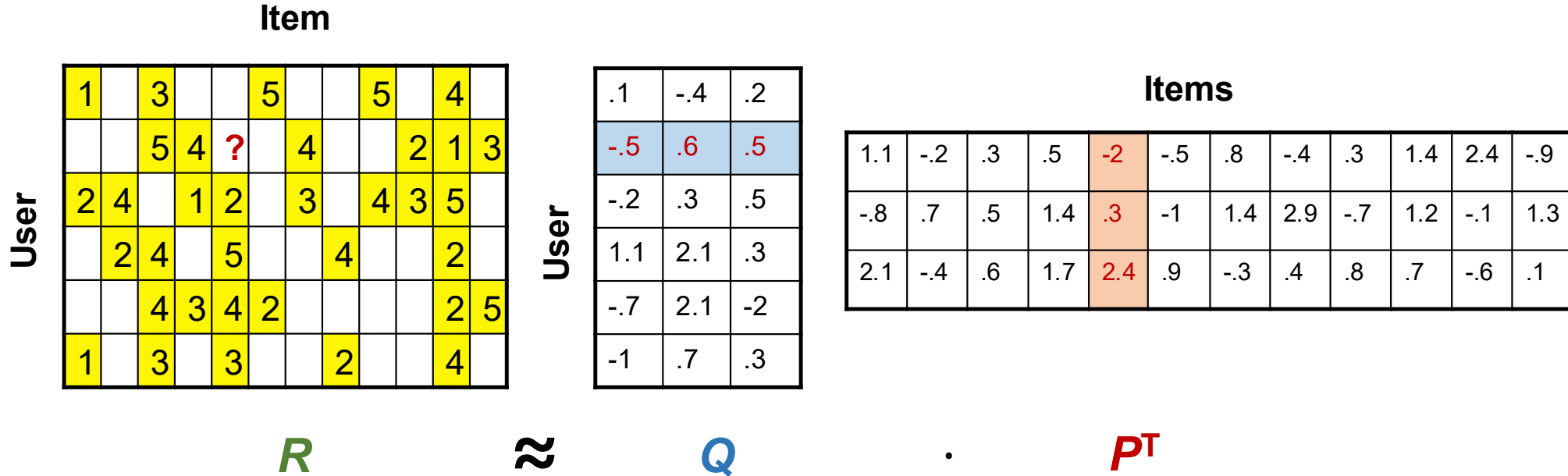


Fig from mmids.org

Latent space of user and item

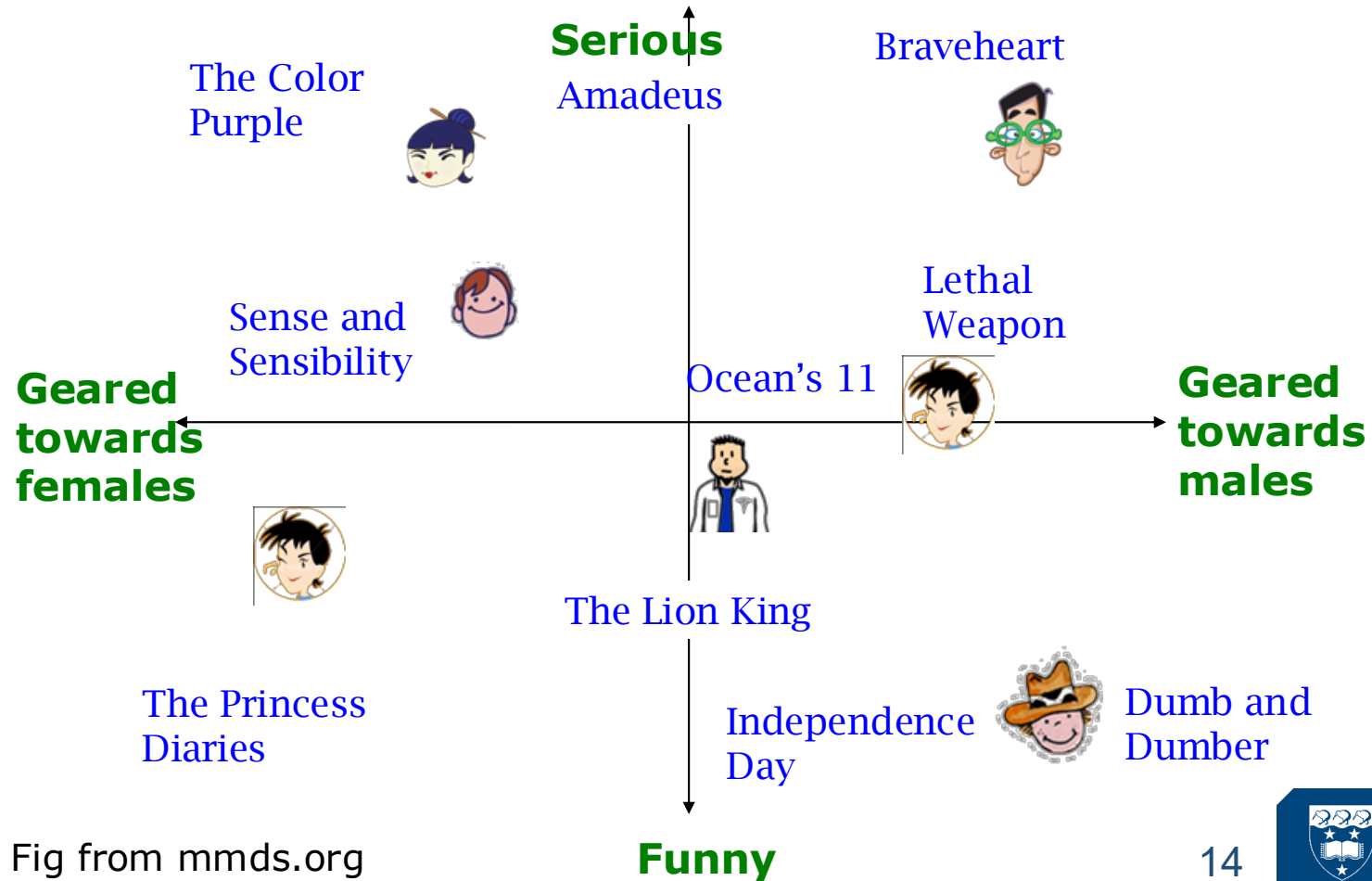


Fig from mmds.org

Singular value decomposition (SVD)

$$\begin{array}{c} \mathbf{R} \\ \left[\begin{array}{ccc} r_{11} & \cdots & r_{1m} \\ \vdots & \ddots & \vdots \\ r_{n1} & \cdots & r_{nm} \end{array} \right] \\ n \times m \end{array} \approx \begin{array}{c} \mathbf{U} \\ \left[\begin{array}{ccc} u_{11} & \cdots & u_{1k} \\ \vdots & \ddots & \vdots \\ u_{n1} & \cdots & u_{nk} \end{array} \right] \\ n \times k \end{array} \begin{array}{c} \mathbf{\Lambda} \\ \left[\begin{array}{ccc} \lambda_{11} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \lambda_{kk} \end{array} \right] \\ k \times k \end{array} \begin{array}{c} \mathbf{V}^T \\ \left[\begin{array}{ccc} v_{11} & \cdots & v_{1m} \\ \vdots & \ddots & \vdots \\ v_{k1} & \cdots & v_{km} \end{array} \right] \\ k \times m \end{array}$$

- $\mathbf{R}$ – rating matrix
- $\mathbf{U}$ – user preference on (latent) factors
- $\mathbf{\Lambda}$ – singular values, importance of the factors
- $\mathbf{V}$ – item quality on (latent) factors

$$\mathbf{R} \approx \mathbf{Q}\mathbf{P}^T, \quad \mathbf{Q} = \mathbf{U}, \quad \mathbf{P}^T = \mathbf{\Lambda} \mathbf{V}^T$$

Singular value decomposition (SVD)

- Properties of SVD $R = U\Lambda V^T$:
 - U and V are orthogonal matrices:

$$U^T U = I \text{ and } V^T V = I$$

- Each column of U is an eigenvector of RR^T

$$U\Lambda V^T V\Lambda U^T = U\Lambda^2 U^T$$

- Each column of V is an eigenvector of $R^T R$

$$V\Lambda U^T U\Lambda V^T = V\Lambda^2 V^T$$

Matrix factorization

- SVD is both user-based and item-based

R – $n \times m$ rating matrix

$$R = U\Lambda V^T \Rightarrow \begin{cases} RR^T = U\Lambda^2 U^T \\ R^T R = V\Lambda^2 V^T \end{cases}$$



U and V are in latent space
that encodes similarities

$$RR^T = \begin{bmatrix} \sum_j r_{1j}r_{1j} & \cdots & \sum_j r_{1j}r_{nj} \\ \vdots & \ddots & \vdots \\ \sum_j r_{nj}r_{1j} & \cdots & \sum_j r_{nj}r_{nj} \end{bmatrix}$$

$n \times n$

User dot-product matrix

$$R^T R = \begin{bmatrix} \sum_i r_{i1}r_{i1} & \cdots & \sum_i r_{i1}r_{im} \\ \vdots & \ddots & \vdots \\ \sum_i r_{im}r_{i1} & \cdots & \sum_i r_{im}r_{im} \end{bmatrix}$$

$m \times m$

Item dot-product matrix

Solving SVD as Optimization

- Minimize the Sum of Squared Errors

$$\min_{U, \Lambda, \Sigma} \sum_{i \in U, j \in P} (r_{ij} - [U\Lambda V^T]_{ij})^2$$

- Pros:
 - It directly optimizes the SSE and thus minimize the RMSE
- Cons:
 - SVD needs to run on all the entries!
 - Missing values will also be used to calculate the loss!

Latent factor model

- A special way to solve the matrix factorization problem by only counting the loss on **available ratings**!

$$\min_{P,Q} \sum_{r_{ij} \in \mathbf{R}} (r_{ij} - \mathbf{q}_i^T \cdot \mathbf{p}_j)^2$$

- In this model, we aim to predict ratings

$$\hat{r}_{ij} = \mathbf{q}_i^T \cdot \mathbf{p}_j$$

- Additional merit - it is easy to incorporate the bias terms!

$$\hat{r}_{ij} = b_g + b_i + b_j + \mathbf{q}_i^T \cdot \mathbf{p}_j$$

Finding latent factors

- Our goal:

$$\min_{P,Q} \sum_{r_{ij} \in R} (r_{ij} - \mathbf{q}_i^T \cdot \mathbf{p}_j)^2$$

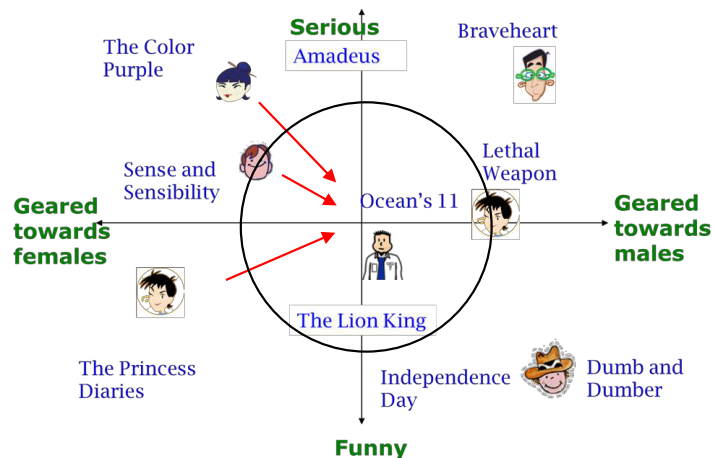
- The number of factors k
 - Too large? – overfitting on the limited data, i.e., the model fits perfectly to the current data, but may perform poor on unseen data.
 - Too small? – not precise in prediction.

Regularization

- Objective with **Regularization**:

$$\min_{P,Q} \underbrace{\sum_{r_{ij} \in R} (r_{ij} - \mathbf{q}_i^T \cdot \mathbf{p}_j)^2}_{\text{SSE}} + \underbrace{\lambda_1 \sum_{i \in U} \|\mathbf{q}_i\|^2 + \lambda_2 \sum_{j \in P} \|\mathbf{p}_j\|^2}_{\text{Length of the vectors}}$$

λ_1, λ_2 are the hyperparameters to balance SSE and the regularization terms



Effect:
Move the latent vectors towards the origin

Stochastic Gradient Descent

- Our loss function:

$$\mathcal{L} = \sum_{r_{ij} \in R} (r_{ij} - \mathbf{q}_i^T \cdot \mathbf{p}_j)^2 + \lambda_1 \sum_{i \in U} \|\mathbf{q}_i\|^2 + \lambda_2 \sum_{j \in P} \|\mathbf{p}_j\|^2$$

- Stochastic Gradient Decent (SGD)**

- Initialize P, Q and all the bias terms randomly
- Do stochastic gradient descent:

- $\mathbf{q}_{if}^{(t+1)} \leftarrow \mathbf{q}_{if}^{(t)} - \eta \nabla_{\mathbf{q}_{if}} \mathcal{L}_{ij}^{(t)}$

$$\nabla_{\mathbf{q}_{if}} \mathcal{L}_{ij}^{(t)} = -2 \left(r_{ij} - \mathbf{q}_i^{T(t)} \cdot \mathbf{p}_j^{(t)} \right) \mathbf{p}_{jf}^{(t)} + 2\lambda_1 \mathbf{q}_{if}^{(t)}$$

The f -th factor of in the latent vector q_i

- $\mathbf{p}_{jf}^{(t+1)} \leftarrow \mathbf{p}_{jf}^{(t)} - \eta \nabla_{\mathbf{p}_{jf}} \mathcal{L}_{ij}^{(t)}$

$$\nabla_{\mathbf{p}_{jf}} \mathcal{L}_{ij}^{(t)} = -2 \left(r_{ij} - \mathbf{q}_i^{T(t)} \cdot \mathbf{p}_j^{(t)} \right) \mathbf{q}_{if}^{(t)} + 2\lambda_2 \mathbf{p}_{jf}^{(t)}$$

The f -th factor of in the latent vector p_j

Incorporating bias to LFM

- Consider predicting the rating of a user i to an item j with bias:

$$\hat{r}_{ij} = b_g + \underbrace{b_i^{(user)} + b_j^{(item)}}_{b_{ij}} + \mathbf{q}_i^T \cdot \mathbf{p}_j$$

- Our goal:

$$\min_{P, Q, b_i^{(user)}, b_j^{(item)}} \sum_{r_{ij} \in R} (r_{ij} - \mathbf{q}_i^T \cdot \mathbf{p}_j - b_{ij})^2$$

Note: The bias terms $b_i^{(user)}, b_j^{(item)}$ can be learned from the data directly.

Regularization

- Objective with **Regularization**:

$$\min_{P, Q, b_i, b_j} \underbrace{\sum_{r_{ij} \in R} (r_{ij} - \mathbf{q}_i^T \cdot \mathbf{p}_j - b_{ij})^2}_{\text{SSE}}$$
$$+ \underbrace{\lambda_1 \sum_{i \in U} \|\mathbf{q}_i\|_2^2 + \lambda_2 \sum_{j \in P} \|\mathbf{p}_j\|_2^2 + \lambda_3 \sum_{i \in U} (b_i^{(user)})^2 + \lambda_4 \sum_{j \in P} (b_j^{(item)})^2}_{\text{Length of the vectors}}$$

$\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are the hyperparameters to balance SSE and the regularization terms

Stochastic Gradient Descent

- Our loss function:

$$\mathcal{L} = \sum_{r_{ij} \in R} \left(r_{ij} - \mathbf{q}_i^T \mathbf{p}_j - b_g - b_i^{(user)} - b_j^{(item)} \right)^2 + \lambda_1 \sum_{i \in U} \|\mathbf{q}_i\|^2 + \lambda_2 \sum_{j \in P} \|\mathbf{p}_j\|^2 + \lambda_3 \sum_{i \in U} \left(b_i^{(user)} \right)^2 + \lambda_4 \sum_{j \in P} \left(b_j^{(item)} \right)^2$$

- Stochastic Gradient Decent (SGD)**

- Initialize P, Q and all the bias terms randomly
- Run SGD for each rating:

- Compute the predicted rating: $\hat{r}_{ij}^{(t)} = \mathbf{q}_i^{(t)T} \mathbf{p}_j^{(t)} + b_g + b_i^{(user)(t)} + b_j^{(item)(t)}$
- $\mathbf{q}_{if}^{(t+1)} \leftarrow \mathbf{q}_{if}^{(t)} - \eta \nabla_{\mathbf{q}_{if}} \mathcal{L}_{ij}^{(t)} = \mathbf{q}_{if}^{(t)} - \eta [-2 (r_{ij} - \hat{r}_{ij}^{(t)}) \mathbf{p}_{jf}^{(t)} + 2\lambda_1 \mathbf{q}_{if}^{(t)}]$
- $\mathbf{p}_{jf}^{(t+1)} \leftarrow \mathbf{p}_{jf}^{(t)} - \eta \nabla_{\mathbf{p}_{jf}} \mathcal{L}_{ij}^{(t)} = \mathbf{p}_{jf}^{(t)} - \eta [-2 (r_{ij} - \hat{r}_{ij}^{(t)}) \mathbf{q}_{if}^{(t)} + 2\lambda_2 \mathbf{p}_{jf}^{(t)}]$
- $b_i^{(user)(t+1)} \leftarrow b_i^{(user)(t)} - \eta \nabla_{b_i} \mathcal{L}_{ij}^{(t)} = b_i^{(t)} - \eta [-2 (r_{ij} - \hat{r}_{ij}^{(t)}) + 2\lambda_3 b_i^{(user)(t)}]$
- $b_j^{(item)(t+1)} \leftarrow b_j^{(item)(t)} - \eta \nabla_{b_j} \mathcal{L}_{ij}^{(t)} = b_j^{(t)} - \eta [-2 (r_{ij} - \hat{r}_{ij}^{(t)}) + 2\lambda_4 b_j^{(item)(t)}]$

Summary

- Content-based methods
 - Using item attributes and text content to profile items and users
- Collaborative filtering
 - User-based & Item-based – k-nearest neighbors
 - Interpolation method
 - Matrix factorization and Latent factor model
- Evaluation of recommender systems